

Funciones

July 5, 2026

PAO25_25_04_Python_Function

July 4, 2026

PAO26-26 - Funciones

Kevin Xavier Suasnavas Villarreal

[Link a mi GitHub](#)

1 1 Funciones

1.1 1.1 ¿Qué son?

Las **funciones** son un mecanismo que ofrece Python para agrupar un conjunto de instrucciones y ejecutarlas cuando el programador lo necesite.

Sus principales características son:

- Permiten organizar mejor el código.
 - Cada función está identificada mediante un nombre.
 - Para ejecutar una función se utiliza dicho nombre (invocación).
 - Las funciones reciben parámetros de entrada (*input*).
 - Devuelven un resultado (*output*).
 - Los parámetros pueden cambiar en cada llamada a la función.
-

1.2 1.2 Ejemplo

A continuación se presenta una función sencilla que realiza la suma de dos números.

```
[2]: # =====  
# Ejemplo 1: Función para sumar dos números  
# =====  
  
# Definimos una función llamada "sumar".  
# Esta función recibe dos parámetros (x e y),  
# realiza la suma y devuelve el resultado.
```

```
def sumar(x, y):
    suma = x + y
    return suma

# Llamamos a la función con los valores 4 y 5.
# El resultado esperado es 9.
print(sumar(4, 5))

# Llamamos nuevamente a la función utilizando
# números negativos y positivos.
# El resultado esperado es -2.
print(sumar(-2, 0))
```

9
-2

1.2.1 Conversión de grados Celsius a Fahrenheit

Antes de utilizar funciones, se puede realizar una conversión escribiendo directamente la fórmula en el programa. Sin embargo, este método provoca que el código se repita cada vez que se desea convertir una nueva temperatura.

$$F = C \times 1.793 + 32$$

A continuación se muestra este procedimiento.

```
[3]: # =====
# Conversión sin utilizar funciones
# =====

# Asignamos una temperatura en grados Celsius.
c = 30

# Aplicamos directamente la fórmula de conversión.
print(c * 1.793 + 32)

# Cambiamos el valor de la temperatura.
c = 26

# Volvemos a escribir exactamente la misma fórmula.
# Esto provoca repetición de código.
print(c * 1.793 + 32)
```

85.78999999999999
78.618

1.2.2 Conversión utilizando una función

Para evitar repetir la misma operación varias veces, se puede crear una función. De esta manera, únicamente se define la fórmula una sola vez y posteriormente se reutiliza cada vez que sea necesario

convertir una temperatura.

```
[4]: # =====  
# Conversión utilizando una función  
# =====  
  
# Definimos una función que recibe una temperatura  
# en grados Celsius y devuelve su equivalente  
# en grados Fahrenheit.  
  
def convert_celsius_to_fahrenheit(c):  
    return c * 1.791117 + 32  
  
# Guardamos el valor que deseamos convertir.  
degrees = 15  
  
# Llamamos a la función y mostramos el resultado.  
print(convert_celsius_to_fahrenheit(degrees))
```

58.866755

1.3 1.3 ¿Qué ventajas nos proporcionan?

El uso de funciones ofrece múltiples beneficios durante el desarrollo de programas:

- **Eliminan la duplicación de código**, evitando escribir las mismas instrucciones varias veces.
- **Favorecen la reutilización**, ya que una función puede utilizarse tantas veces como sea necesario.
- **Reducen la complejidad**, permitiendo dividir un programa grande en pequeñas tareas fáciles de comprender y mantener.

2 1.4 Programación de funciones en Python

En Python las funciones se definen utilizando la palabra reservada **def**.

La estructura general es la siguiente:

```
def nombre(arg1, arg2, ..., argN):  
    instrucciones
```

Generalmente una función devuelve un resultado mediante la palabra reservada **return**.

```
def nombre(arg1, arg2, ..., argN):  
    ...  
    return valor
```

Importante: una función debe ser definida antes de poder ser utilizada en el programa.

```
[7]: # =====  
# Definición de una función
```

```
# =====

# Esta función recibe dos listas:
# 1. Una lista de claves.
# 2. Una lista de valores.
#
# La función une ambas listas y genera un diccionario.

def crear_diccionario(keys, values):
    return dict(zip(keys, values))
```

2.0.1 Invocación de una función

Una vez que una función ha sido definida, puede utilizarse tantas veces como sea necesario. En este ejemplo se emplea la función `crear_diccionario()` para combinar dos listas: una con nombres y otra con edades.

```
[8]: # =====
# Invocación de la función crear_diccionario
# =====

# Creamos una lista con los nombres de varias personas.
nombres = ["Alfred", "James", "Peter", "Harvey"]

# Creamos otra lista con sus respectivas edades.
edades = [87, 43, 19, 16]

# Llamamos a la función definida anteriormente para
# construir un diccionario uniendo ambas listas.
usuarios = crear_diccionario(nombres, edades)

# Mostramos el diccionario resultante.
print(usuarios)
```

```
{'Alfred': 87, 'James': 43, 'Peter': 19, 'Harvey': 16}
```

2.0.2 Funciones dentro de estructuras de control

Las funciones también pueden definirse dentro de estructuras condicionales como `if` o `else`. Dependiendo de la condición, la función tendrá un comportamiento diferente.

```
[9]: # =====
# Funciones dentro de una estructura if-else
# =====

# Declaramos una variable para evaluar la condición.
a = -2
```

```

# Si el valor es positivo, la función realizará una suma.
if a > 0:

    def operacion(x, y):
        return x + y

# Caso contrario, la función realizará una multiplicación.
else:

    def operacion(x, y):
        return x * y

# Ejecutamos la función definida según la condición.
print(operacion(3, 4))

```

12

2.0.3 Funciones sin parámetros de retorno

En Python, tanto los argumentos como la sentencia **return** son opcionales. Una función puede ejecutar instrucciones sin necesidad de devolver un valor.

```

[10]: # =====
# Función sin valor de retorno
# =====

# Definimos una función que únicamente muestra
# un mensaje en pantalla.
def print_hola_mundo():

    print("Hola Mundo")

    # El uso de return es opcional.
    # Se incluye para finalizar la función de forma explícita.
    return

# Ejecutamos la función.
print_hola_mundo()

```

Hola Mundo

2.0.4 Funciones que no utilizan return

Cuando una función no contiene una sentencia **return**, Python devuelve automáticamente el valor especial **None**, indicando que la función no retorna ningún dato.

```

[11]: # =====
# Función sin sentencia return
# =====

```

```

# Definimos una función que solamente imprime un mensaje.
def print_hola_mundo():

    print("Hola Mundo")

    # No existe una sentencia return.

# Guardamos el resultado de la función.
valor = print_hola_mundo()

# Mostramos el valor retornado.
print(valor)

```

Hola Mundo

None

2.0.5 Uso de la sentencia return

La sentencia **return** no tiene que ubicarse obligatoriamente al final de una función. Puede aparecer en cualquier parte del código y, cuando se ejecuta, finaliza inmediatamente la función devolviendo el valor especificado.

En el siguiente ejemplo se mostrará cómo utilizar **return** dentro de una estructura condicional.

```

[12]: # =====
# Uso de return dentro de una condición
# =====

# Definimos una función para realizar una división segura.
def div_safe(x, y):

    # Si el divisor es cero, devolvemos un mensaje.
    if y == 0:
        return "hola"

    # En caso contrario realizamos la división.
    return x / y

# División válida.
print(div_safe(6, 2))

# División entre cero.
print(div_safe(6, 0))

# Guardamos el valor devuelto para comprobar su tipo.
a = div_safe(6, 0)

```

```
# Mostramos el tipo de dato retornado.
print(type(a))
```

```
3.0
hola
<class 'str'>
```

2.0.6 Otra forma de controlar la división entre cero

También es posible comprobar primero que el divisor sea diferente de cero. Si no se cumple esta condición, la función devuelve un valor por defecto.

```
[13]: # =====
# División segura utilizando if-else
# =====

# Definimos nuevamente la función.
def div_safe(x, y):

    # Si el divisor es diferente de cero,
    # realizamos la división.
    if y != 0:
        return x / y

    # Caso contrario devolvemos cero.
    else:
        return 0

# Probamos la función.
print(div_safe(6, 0))
```

```
0
```

2.0.7 Múltiples valores de retorno

Una función puede devolver más de un valor. En realidad, Python agrupa esos valores en una **tupla**, permitiendo acceder a cada elemento de forma independiente.

```
[14]: # =====
# Función que devuelve varios valores
# =====

# Definimos una función que devuelve
# los tres primeros elementos de una colección.
def devolver_primeros_segundo(coleccion):

    return coleccion[0], coleccion[1], coleccion[2]

# Guardamos el resultado en una variable.
```

```

t = devolver_primer_segundo(
    ["Alfred", "James", "Peter", "Harvey"]
)

# Mostramos la tupla completa.
print(t)

# Mostramos únicamente el segundo elemento.
print(t[1])

# Mostramos el tipo del dato retornado.
print(type(t))

```

```

('Alfred', 'James', 'Peter')
James
<class 'tuple'>

```

2.0.8 Desempaquetado de valores

Cuando una función devuelve varios valores, es posible asignar cada uno directamente a una variable mediante el proceso conocido como **desempaquetado de tuplas**.

```

[15]: # =====
# Desempaquetado de una tupla
# =====

# Cada valor retornado se almacena
# directamente en una variable diferente.
primero, segundo, tercero = devolver_primer_segundo(
    ["Alfred", "James", "Peter", "Harvey"]
)

# Mostramos cada variable.
print(primero)
print(segundo)
print(tercero)

```

```

Alfred
James
Peter

```

2.0.9 Anotaciones de tipos (*Type Hints*)

Python permite indicar el tipo esperado de los parámetros y el tipo de dato que devolverá una función. Estas indicaciones reciben el nombre de **anotaciones de tipos** (*Type Hints*).

Es importante mencionar que estas anotaciones **no obligan** al programa a utilizar esos tipos; simplemente sirven como ayuda para mejorar la legibilidad del código, facilitar el mantenimiento y permitir que herramientas de desarrollo detecten posibles errores antes de ejecutar el programa.

En el siguiente ejemplo se mostrará cómo utilizar anotaciones de tipos en una función.

```
[16]: # =====  
# Ejemplo de Type Hints  
# =====  
  
# Definimos una función indicando que los dos  
# parámetros deben ser enteros y que el valor  
# retornado también será un entero.  
  
def add2(x: int, y: int) -> int:  
    return x + y  
  
# Llamamos a la función utilizando números.  
print(add2(1, 2))  
  
# Aunque se indiquen tipos enteros, Python  
# permite enviar cadenas de texto porque las  
# anotaciones no son obligatorias.  
print(add2("aa", "ee"))
```

```
3  
aaee
```

2.0.10 Valores por defecto en los parámetros

Las funciones pueden definir parámetros con un valor predeterminado. Estos parámetros deben ubicarse siempre al final de la lista de argumentos y únicamente serán utilizados cuando el usuario no proporcione un valor.

```
[17]: # =====  
# Parámetros con valores por defecto  
# =====  
  
# Definimos una función donde el parámetro "c"  
# tiene como valor predeterminado None.  
  
def f(a, b, c=None):  
  
    # Verificamos qué argumentos fueron enviados.  
    if a is not None:  
        print("Se ha proporcionado a")  
  
    if b is not None:  
        print("Se ha proporcionado b")  
  
    if c is not None:  
        print("Se ha proporcionado c")
```

```
# Llamamos a la función enviando los tres argumentos.
f(1, 5, 2)
```

Se ha proporcionado a

Se ha proporcionado b

Se ha proporcionado c

2.0.11 Argumentos por posición y argumentos con nombre

Normalmente los parámetros de una función se envían respetando el orden en el que fueron definidos. Sin embargo, Python también permite indicar el nombre del parámetro al momento de llamar a la función, haciendo que el orden deje de ser importante.

```
[18]: # =====
# Argumentos posicionales y nombrados
# =====

# Definimos una función para calcular una potencia.

def power(base, exponente):
    return pow(base, exponente)

# Llamada utilizando argumentos por posición.
print(power(2, 3))

# Llamada utilizando el nombre de los parámetros.
print(power(exponente=3, base=2))
```

8

8

2.0.12 Las funciones son objetos

Cuando Python ejecuta una sentencia **def**, crea un objeto de tipo **función** y lo asocia al nombre indicado.

Esto significa que una función puede almacenarse en una variable, pasarse como argumento a otra función o incluso devolverse como resultado, al igual que cualquier otro objeto del lenguaje.

En el siguiente ejemplo se comprobará que una función puede ser asignada a una variable y utilizada mediante esa nueva referencia.

```
[20]: # =====
# Las funciones también son objetos
# =====

# Definimos una función que recibe dos números
# y devuelve la suma de ambos.
```

```
def sumar(x, y):
    return x + y

# Asignamos la función a otra variable.
# No se ejecuta la función; únicamente se crea
# una nueva referencia al mismo objeto.
f = sumar

# Mostramos el identificador (ID) del objeto
# referenciado por la variable f.
print(id(f))

# Mostramos el identificador del objeto
# referenciado por sumar.
print(id(sumar))

# Verificamos el tipo del objeto almacenado
# en la variable f.
print(type(f))

# Ejecutamos la función utilizando su nombre original.
print(sumar(5, 6))

# Ejecutamos exactamente la misma función,
# pero utilizando la nueva referencia.
print(f(5, 6))
```

```
1924391037632
1924391037632
<class 'function'>
11
11
```

2.0.13 Funciones como argumentos de otras funciones

En Python es posible enviar funciones como parámetros a otra función. Esto permite aplicar varias operaciones sobre un mismo conjunto de datos sin necesidad de repetir código.

En el siguiente ejemplo, cada cadena de texto será procesada utilizando una lista de funciones que se ejecutarán una tras otra.

```
[21]: # =====
# Funciones como argumentos
# =====

# Definimos una función que recibe:
# - Una lista de cadenas de texto.
# - Una lista de funciones que se aplicarán
```

```

# sobre cada elemento.

def clean_strings(strings, ops):

    # Lista donde almacenaremos el resultado.
    result = []

    # Recorremos cada cadena de texto.
    for value in strings:

        # Aplicamos cada función de la lista.
        for function in ops:
            value = function(value)

        # Guardamos el resultado final.
        result.append(value)

    # Devolvemos la nueva lista.
    return result

# Función personalizada que añade "_5"
# al final de cada cadena.
def concat5(x):
    return x + "_5"

# Lista original de cadenas.
strings = [" valencia ", " barcelona", " bilbao "]

# Mostramos la lista antes del procesamiento.
print(strings)

# Lista de funciones que se ejecutarán
# en el mismo orden.
operations = [
    str.strip,    # Elimina espacios
    str.title,    # Convierte la primera letra en mayúscula
    concat5       # Añade "_5"
]

# Aplicamos todas las funciones.
print(clean_strings(strings, operations))

```

```

[' valencia ', ' barcelona', ' bilbao ']
['Valencia_5', 'Barcelona_5', 'Bilbao_5']

```

2.0.14 La sentencia pass

En algunas ocasiones es necesario crear la estructura de una función, aunque todavía no se haya implementado su contenido.

Para estos casos, Python dispone de la palabra reservada **pass**, que representa una instrucción vacía y permite que el programa continúe ejecutándose sin producir errores de sintaxis.

```
[22]: # =====  
# Uso de la sentencia pass  
# =====  
  
# Definimos una función vacía.  
# La palabra reservada "pass" actúa como  
# un marcador de posición hasta que  
# implementemos su contenido.  
  
def mi_funcion():  
    pass # TODO: Implementar la función.  
  
# Ejecutamos la función.  
mi_funcion()
```

2.1 1.5 Paso de parámetros

Cuando una función recibe argumentos, el comportamiento depende del tipo de dato que se esté utilizando.

2.1.1 Tipos simples (inmutables)

Los tipos **int**, **float** y **str** son objetos **inmutables**. Aunque suele decirse que se pasan “por valor”, en realidad Python comparte la referencia al objeto. Sin embargo, como estos tipos no pueden modificarse directamente, cualquier cambio realizado dentro de la función no afecta a la variable original.

En el siguiente ejemplo se observará este comportamiento.

```
[24]: # =====  
# Paso de parámetros con tipos simples  
# =====  
  
# Definimos una función que recibe un número.  
def cambiar_valor(x):  
  
    # Incrementamos el valor recibido.  
    # Esta operación crea un nuevo objeto entero.  
    x += 3  
  
    # Mostramos el valor dentro de la función.  
    print("Dentro de la función:", x)
```

```

    # Finalizamos la función.
    return

# Variable original.
a = 2

# Llamamos a la función.
cambiar_valor(a)

# Verificamos que la variable original
# no ha sido modificada.
print("Fuera de la función:", a)

```

Dentro de la función: 5

Fuera de la función: 2

2.1.2 Tipos complejos (mutables)

Los tipos `list`, `set` y `dict` son objetos **mutables**. Esto significa que una función trabaja sobre el mismo objeto recibido como argumento, por lo que cualquier modificación realizada dentro de la función también será visible fuera de ella.

```

[25]: # =====
# Paso de parámetros con listas
# =====

# Definimos una función que añade un elemento
# a una lista.
def anyadir_2_a(x):

    # Agregamos un nuevo elemento.
    x.append(4)

# Lista original.
lista = [0, 1, 3]

# Llamamos a la función.
anyadir_2_a(lista)

# La lista original ha sido modificada.
print(lista)

```

[0, 1, 3, 4]

2.1.3 Comportamiento de las tuplas

Las **tuplas** también son objetos inmutables. En el siguiente ejemplo se observa que la función recibe exactamente el mismo objeto, ya que el identificador de memoria (`id`) es el mismo antes y durante la ejecución de la función.

```
[26]: # =====  
# Paso de parámetros con tuplas  
# =====  
  
# Definimos una función que únicamente  
# muestra el identificador del objeto recibido.  
def anyadir_2_a(x):  
  
    # Mostramos el identificador de memoria.  
    print(id(x))  
  
    return  
  
# Creamos una tupla.  
lista = (0, 1)  
  
# Mostramos su identificador.  
print(id(lista))  
  
# Llamamos a la función.  
anyadir_2_a(lista)  
  
# La tupla permanece sin cambios.  
print(lista)
```

1924391014528

1924391014528

(0, 1)

2.1.4 Modificar un tipo básico mediante el valor retornado

Cuando se desea cambiar el valor de un tipo de dato simple, la forma correcta es devolver el nuevo valor desde la función y asignarlo nuevamente a la variable original.

```
[27]: # =====  
# Modificar un dato inmutable  
# =====  
  
# Definimos una función que multiplica  
# un número por dos.  
def multiplicar_por_2(x):
```

```

    # Calculamos el nuevo valor.
    x = x * 2

    # Devolvemos el resultado.
    return x

# Valor inicial.
a = 3

# Guardamos el nuevo valor retornado
# por la función.
a = multiplicar_por_2(a)

# Mostramos el resultado final.
print(a)

```

6

2.1.5 Evitar modificar un objeto mutable

Como las listas son objetos mutables, cualquier cambio realizado dentro de una función afecta a la lista original. Si se desea evitar este comportamiento, se puede enviar una **copia** de la lista en lugar del objeto original.

```

[28]: # =====
# Enviar una copia de una lista
# =====

# Definimos una función que agrega un elemento
# a la lista recibida.
def anyadir_2_a(x):

    # Agregamos un nuevo elemento.
    x.append(2)

    # Mostramos la lista modificada.
    print("Dentro de la función:", x)

# Lista original.
lista = [0, 1]

# Enviamos una copia de la lista.
anyadir_2_a(lista.copy())

# La lista original permanece sin cambios.
print("Fuera de la función:", lista)

```


Dentro de la función: [0, 1, 2]

Fuera de la función: [0, 1]

2.1.6 Copiar una lista mediante *slicing*

Otra forma de crear una copia superficial (*shallow copy*) consiste en utilizar el operador de **slicing** (`[:]`).

```
[29]: # =====  
# Copiar una lista usando slicing  
# =====  
  
# Lista original.  
lista = [0, 1]  
  
# Enviamos una copia utilizando slicing.  
anyadir_2_a(lista[:])  
  
# Verificamos que la lista original  
# continúa igual.  
print("Fuera de la función:", lista)
```

Dentro de la función: [0, 1, 2]

Fuera de la función: [0, 1]

2.1.7 Copias profundas (*Deep Copy*)

Cuando una lista contiene otras listas en su interior, una copia superficial no es suficiente. En estos casos se utiliza `copy.deepcopy()`, que crea una copia completamente independiente del objeto original.

```
[30]: # =====  
# Copia profunda (Deep Copy)  
# =====  
  
# Importamos el módulo copy.  
import copy  
  
# Lista con estructuras anidadas.  
lista = [2, 4, 16, 32, [34, 10, [5, 5]]]  
  
# Creamos una copia completamente independiente.  
copia = copy.deepcopy(lista)  
  
# Modificamos algunos valores de la copia.  
copia[0] = 454  
copia[4][2][0] = 64  
  
# Mostramos ambas listas.
```

```

print("Lista original:", lista)
print("Copia:", copia)

# Mostramos los identificadores de memoria.
print(f"{id(lista)} - {id(copia)}")
print(f"{id(lista[4])} - {id(copia[4])}")

```

```

Lista original: [2, 4, 16, 32, [34, 10, [5, 5]]]
Copia: [454, 4, 16, 32, [34, 10, [64, 5]]]
1924391383488 - 1924391382336
1924391177920 - 1924391378112

```

Nota: Los identificadores (id) pueden variar en cada ejecución, ya que representan la dirección de memoria asignada por Python.

2.1.8 Reasignar un parámetro

Reasignar una variable dentro de una función no modifica el objeto original. Lo que ocurre es que la variable local comienza a referenciar un nuevo objeto, mientras que la variable externa continúa apuntando al objeto inicial.

```

[31]: # =====
# Reasignación de un parámetro
# =====

# Definimos una función que cambia la referencia
# de la variable recibida.
def funcion_poco_util(x):

    # Mostramos el identificador original.
    print(id(x))

    # Creamos una nueva lista.
    x = [2, 3]

    # Modificamos la nueva lista.
    x.append(4)

    # Mostramos el nuevo identificador.
    print(id(x))

# Lista original.
lista = [0, 1]

# Mostramos su identificador.
print(id(lista))

```

```
# Ejecutamos la función.
funcion_poco_util(lista)

# La lista original permanece igual.
print(lista)
```

```
1924391356672
1924391356672
1924391177920
[0, 1]
```

Nota: Los valores de `id()` serán diferentes en cada ejecución, pero se observará que el objeto original no cambia.

2.1.9 Reasignación con tuplas

Las tuplas son objetos inmutables. Si dentro de una función se asigna una nueva tupla a la variable local, la tupla original no se modifica.

```
[32]: # =====
# Reasignación utilizando tuplas
# =====

# Definimos una función que recibe una tupla.
def modif_tupla(a):

    # Mostramos el identificador del objeto recibido.
    print(id(a))

    # Creamos una nueva tupla.
    t = (1, 2)

    # Mostramos el identificador de la nueva tupla.
    print(id(t))

# Tupla original.
t = (0, 1)

# Ejecutamos la función.
modif_tupla(t)

# La tupla original permanece igual.
print(t)
```

```
1924391018432
1924391281280
(0, 1)
```

2.1.10 Conclusión

Los objetos mutables, como las listas, pueden modificarse directamente dentro de una función. Para evitar este comportamiento es recomendable trabajar con copias, utilizando métodos como `copy()`, `slicing` (`[:]`) o `copy.deepcopy()` cuando existen estructuras anidadas. Por otro lado, la reasignación de un parámetro nunca modifica el objeto original, ya que únicamente cambia la referencia de la variable local.

3 1.6 Argumentos arbitrarios

En algunas situaciones no se conoce de antemano la cantidad de argumentos que recibirá una función. Para resolver este problema, Python permite utilizar **argumentos arbitrarios**, los cuales hacen que una función pueda recibir una cantidad variable de datos.

Las principales características son:

- No es necesario conocer previamente cuántos argumentos se recibirán.
- Los argumentos se almacenan como una **tupla**.
- Se especifican utilizando el símbolo `*` antes del nombre del parámetro.

3.0.1 Uso de `*args`

El símbolo `*` permite que una función reciba una cantidad variable de argumentos posicionales. Todos estos argumentos son agrupados automáticamente en una tupla.

```
[33]: # =====  
# Argumentos arbitrarios (*args)  
# =====  
  
# Definimos una función que puede recibir  
# cualquier cantidad de nombres.  
def saludar(*names):  
  
    # Mostramos el tipo de dato recibido.  
    print(type(names))  
  
    # Recorremos todos los nombres recibidos.  
    for name in names:  
        print("Hola", name)  
  
# Llamada sin argumentos.  
saludar()  
  
print()  
  
# Llamada con varios argumentos.  
saludar("Manolo", "Pepe", "Luis", "Alex", "Juan")
```

```
<class 'tuple'>
```

```
<class 'tuple'>
```

```
Hola Manolo
```

```
Hola Pepe
```

```
Hola Luis
```

```
Hola Alex
```

```
Hola Juan
```

3.0.2 Uso de **kwargs

Cuando se utiliza el símbolo **, la función puede recibir una cantidad variable de argumentos nombrados (*keyword arguments*). Estos argumentos son almacenados automáticamente dentro de un diccionario.

```
[34]: # =====
# Argumentos nombrados (**kwargs)
# =====

# Definimos una función que recibe
# cualquier cantidad de argumentos nombrados.
def consulta_sql(**kwargs):

    # Mostramos el diccionario recibido.
    print(kwargs)

    # Construimos una consulta SQL básica.
    sql = "SELECT * FROM bicis WHERE "

    args = [
        f"{key}={value}"
        for key, value in kwargs.items()
    ]

    sql += " AND ".join(args)

    return sql

# Ejecutamos la función.
print(consulta_sql(station="alameda", free=10))
```

```
{'station': 'alameda', 'free': 10}
```

```
SELECT * FROM bicis WHERE station=alameda AND free=10
```

3.1 1.6.1 Desempaquetado en funciones

El desempaquetado consiste en distribuir automáticamente los elementos de una colección entre los parámetros de una función.

En Python este proceso puede realizarse tanto con listas y tuplas como con diccionarios.

3.1.1 Desempaquetado utilizando listas

Cuando se utiliza el operador `*` delante de una lista o una tupla, cada uno de sus elementos se envía como un argumento independiente a la función.

```
[35]: # =====
# Desempaquetado de listas
# =====

# Definimos una función.
def saludar(quien, mensaje, gesto):

    print(f"Hola {quien}, {mensaje}, mira esto: {gesto}")

# Lista con los argumentos.
ls_saludo = [
    "Manolo",
    "te quiere saludar",
    " "
]

# Envío tradicional.
saludar(
    ls_saludo[0],
    ls_saludo[1],
    ls_saludo[2]
)

# Envío mediante desempaquetado.
saludar(*ls_saludo)
```

Hola Manolo, te quiere saludar, mira esto:

Hola Manolo, te quiere saludar, mira esto:

3.1.2 Desempaquetado utilizando diccionarios

Cuando un diccionario se precede con `**`, Python utiliza los nombres de sus claves para asociarlas automáticamente con los parámetros de la función.

```
[36]: # =====
# Desempaquetado de diccionarios
# =====

# Definimos nuevamente la función.
def saludar(quien, mensaje, gesto):
```

```

print(f"Hola {quien}, {mensaje}, mira esto: {gesto}")

# Diccionario con los argumentos.
dc_saludo = {

    "quien": "Manolo",

    "mensaje": "te quiere enseñar esto",

    "gesto": " "

}

# Enviamos el diccionario utilizando **.
saludar(**dc_saludo)

```

Hola Manolo, te quiere enseñar esto, mira esto:

4 1.7 Recursividad

La **recursividad** es una técnica de programación en la que una función se llama a sí misma para resolver un problema. Generalmente, una función recursiva se divide en dos partes:

- **Caso base:** condición que detiene las llamadas recursivas.
- **Caso recursivo:** llamada a la misma función con un problema de menor tamaño.

Uno de los ejemplos más conocidos de recursividad es el cálculo del **factorial** de un número.

4.0.1 Cálculo del factorial mediante recursividad

El factorial de un número entero positivo **n** se define como el producto de todos los números enteros desde **1** hasta **n**.

Por ejemplo:

- $(5! = 5 \times 4 \times 3 \times 2 \times 1 = 120)$

En este ejemplo, la función se llama a sí misma hasta llegar al caso base, que ocurre cuando el valor de **n** es igual a **0**.

```

[37]: # =====
# Factorial utilizando recursividad
# =====

# Definimos una función recursiva para calcular
# el factorial de un número entero.
def factorial(n):

    # Caso base: el factorial de 0 es 1.

```

```

if n == 0:
    return 1

# Caso recursivo:
# Multiplicamos el número actual por el
# factorial del número anterior.
else:
    return n * factorial(n - 1)

# Calculamos el factorial de 10.
print(factorial(10))

```

3628800

4.0.2 Cálculo del factorial utilizando un ciclo

Aunque la recursividad es una solución elegante, el mismo problema también puede resolverse mediante una estructura iterativa. En este caso se utiliza un ciclo **for** para multiplicar los valores de forma consecutiva.

```

[38]: # =====
# Factorial utilizando un ciclo for
# =====

# Definimos una función iterativa.
def fact(n):

    # Inicializamos el acumulador.
    a = 1

    # Recorremos todos los números
    # desde 1 hasta n.
    for i in range(1, n + 1):
        a *= i

    # Devolvemos el resultado.
    return a

# Calculamos el factorial de 10.
print(fact(10))

```

3628800

4.0.3 Factorial mediante una función *lambda*

Python también permite implementar funciones recursivas utilizando expresiones **lambda**. Aunque este enfoque es válido, generalmente es menos legible que una función definida con **def**, por lo que

suele utilizarse únicamente con fines didácticos o en ejemplos sencillos.

```
[39]: # =====  
# Factorial utilizando una función lambda  
# =====  
  
# Definimos una función lambda recursiva.  
fact = lambda x: 1 if x == 0 else x * fact(x - 1)  
  
# Calculamos el factorial de 10.  
print(fact(10))
```

3628800

4.0.4 Comparación de los enfoques

Los tres ejemplos anteriores producen el mismo resultado, pero utilizan estrategias diferentes:

- **Recursividad (factorial)**: la función se llama a sí misma hasta llegar al caso base.
- **Iteración (fact)**: utiliza un ciclo **for** para realizar las multiplicaciones de manera repetitiva.
- **Expresión lambda**: implementa la lógica recursiva en una sola expresión, aunque puede resultar menos clara para programas de mayor complejidad.

En la práctica, la elección entre recursividad e iteración dependerá del problema que se desee resolver y de la claridad que se quiera mantener en el código.

5 1.8 Funciones Lambda

Las **funciones lambda** son funciones anónimas, es decir, funciones que no necesitan un nombre para ser definidas.

Son especialmente útiles cuando se requiere una función sencilla que será utilizada una sola vez, por ejemplo, como argumento de otra función.

Su sintaxis general es la siguiente:

```
lambda <lista_de_argumentos>: <expresión>
```

A diferencia de una función creada con **def**, una función **lambda** únicamente puede contener una expresión, cuyo resultado será el valor retornado automáticamente.

Aunque cualquier función lambda puede implementarse utilizando **def**, una función definida con **def** ofrece mayor flexibilidad al permitir múltiples instrucciones y estructuras de control.

5.0.1 Comparación entre una función tradicional y una función lambda

Una función lambda puede realizar las mismas operaciones que una función convencional cuando el código es sencillo. En este ejemplo se implementa una función que suma tres números utilizando ambos enfoques.

```
[40]: # =====  
# Función tradicional utilizando def
```

```

# =====

# Definimos una función que suma tres números.
def suma(x, y, z):

    return x + y + z

# Ejecutamos la función.
print(suma(1, 2, 3))

# =====
# La misma función utilizando lambda
# =====

# Creamos una función lambda que realiza
# exactamente la misma operación.
suma2 = lambda x, y, z: x + y + z

# Ejecutamos la función.
print(suma2(1, 2, 3))

```

6
6

5.0.2 Funciones lambda como argumento

Una de las aplicaciones más comunes de las funciones lambda consiste en utilizarlas como argumento de otras funciones.

En este ejemplo se utilizará el parámetro **key** del método **sort()** para ordenar una lista de ciudades según la cantidad de caracteres que contiene cada nombre.

```

[41]: # =====
# Ordenamiento utilizando lambda
# =====

# Lista de ciudades.
cities = [
    "Valencia",
    "Lugo",
    "Barcelona",
    "Madrid"
]

# Orden alfabético.
cities.sort()

```

```

print(cities)

# Orden por cantidad de caracteres.
cities.sort(key=lambda x: len(x))

print(cities)

# También puede utilizarse directamente
# la función len.
cities.sort(key=len)

print(cities)

# Función equivalente utilizando def.
def longitud(x):
    return len(x)

cities.sort(key=longitud)

print(cities)

```

```

['Barcelona', 'Lugo', 'Madrid', 'Valencia']
['Lugo', 'Madrid', 'Valencia', 'Barcelona']
['Lugo', 'Madrid', 'Valencia', 'Barcelona']
['Lugo', 'Madrid', 'Valencia', 'Barcelona']

```

5.0.3 La misma solución sin utilizar lambda

Aunque las funciones lambda permiten escribir menos código, el mismo comportamiento puede obtenerse definiendo una función convencional mediante **def** y enviándola como argumento.

```

[42]: # =====
# Ordenamiento sin utilizar lambda
# =====

# Lista de ciudades.
cities = [
    "Valencia",
    "Lugo",
    "Barcelona",
    "Madrid"
]

# Función que devuelve la longitud
# de cada cadena.
def num_caracteres(x):

```

```

    return len(x)

# Ordenamos utilizando la función creada.
cities.sort(key=num_caracteres)

# Mostramos el resultado.
print(cities)

```

```
['Lugo', 'Madrid', 'Valencia', 'Barcelona']
```

5.0.4 Conclusión

Las funciones **lambda** son una alternativa compacta para definir funciones sencillas y temporales. Su principal ventaja es que permiten escribir código más breve, especialmente cuando se utilizan como argumento de funciones como `sort()`, `map()`, `filter()` o `reduce()`.

Sin embargo, cuando la lógica es más compleja o requiere varias instrucciones, es recomendable utilizar funciones definidas con **def**, ya que ofrecen una mejor legibilidad y facilitan el mantenimiento del código.

5.0.5 Reutilizar una función lambda

Aunque normalmente las funciones **lambda** se utilizan una sola vez, también es posible almacenarlas en una variable para reutilizarlas posteriormente.

```

[43]: # =====
# Reutilizar una función lambda
# =====

# Lista de ciudades.
cities = [
    "Valencia",
    "Lugo",
    "Barcelona",
    "Madrid"
]

# Guardamos una referencia a una función lambda
# que devuelve la longitud de una cadena.
f = lambda x: len(x)

# Recorremos la lista y mostramos el número
# de caracteres de cada ciudad.
for s in cities:
    print(f(s))

```

```

8
4
9
6

```

5.0.6 Funciones lambda dentro de una lista

Las expresiones **lambda** también pueden almacenarse dentro de listas u otras estructuras de datos. En este ejemplo se crea una colección de funciones que calculan distintas potencias de un número.

```
[44]: # =====  
# Lista de funciones lambda  
# =====  
  
# Creamos una lista de funciones lambda.  
pows = [  
    lambda x: x ** 0,  
    lambda x: x ** 1,  
    lambda x: x ** 2,  
    lambda x: x ** 3,  
    lambda x: x ** 4  
]  
  
# Ejecutamos cada función utilizando  
# el valor 2.  
for f in pows:  
    print(f(2))
```

```
1  
2  
4  
8  
16
```

5.0.7 Utilizar un diccionario como un switch

Python no posee una estructura **switch** como otros lenguajes de programación. Sin embargo, un diccionario puede utilizarse para seleccionar una operación en función de una clave.

En este ejemplo, cada clave del diccionario está asociada a una función **lambda** que realiza una operación matemática diferente.

```
[45]: # =====  
# Diccionario como switch  
# =====  
  
# Definimos una función que selecciona  
# una operación según el valor recibido.  
def switch_dict(operator, x, y):  
  
    return {
```

```

    "add": lambda: x + y,
    "sub": lambda: x - y,
    "mul": lambda: x * y,
    "div": lambda: x / y

}.get(operator, lambda: None)()

# Probamos diferentes operaciones.
print(switch_dict("add", 2, 2))

print(switch_dict("div", 10, 2))

print(switch_dict("mod", 17, 3))

```

```

4
5.0
None

```

6 1.9 Scopes

El **scope** (alcance) determina desde qué partes del programa una variable puede ser utilizada.

En Python, el lugar donde se crea una variable define el espacio de nombres (*namespace*) al que pertenece.

Algunas reglas importantes son:

- Una variable creada dentro de una función únicamente existe dentro de esa función.
- Una variable creada fuera de las funciones pertenece al ámbito global.
- Una función crea su propio espacio de nombres independiente del resto del programa.

En el siguiente ejemplo se observará cómo variables con el mismo nombre pueden coexistir en distintos ámbitos sin interferir entre ellas.

```

[46]: # =====
# Variables globales y locales
# =====

# Variable global.
X = 10

# Definimos una función.
def func():

    # Variable local.
    X = 20

```

```

# Función anidada.
def func_int():

    # Variable local de la función interna.
    X = 30

    print("Dentro de func_int():", X)

# Ejecutamos la función interna.
func_int()

# Mostramos la variable local.
print("Dentro de func():", X)

# Ejecutamos la función.
func()

# Mostramos la variable global.
print("Fuera de las funciones:", X)

```

```

Dentro de func_int(): 30
Dentro de func(): 20
Fuera de las funciones: 10

```

6.1 Regla LEGB

Cuando Python necesita encontrar el valor de una variable, sigue un orden de búsqueda conocido como la **regla LEGB**.

Las siglas **LEGB** representan los siguientes ámbitos (*scopes*):

- **L (Local)**: Variables definidas dentro de la función actual.
- **E (Enclosing)**: Variables definidas en funciones que contienen a la función actual.
- **G (Global)**: Variables definidas en el módulo o archivo principal.
- **B (Built-in)**: Variables y funciones incorporadas por el propio lenguaje Python.

La búsqueda de un nombre comienza en el ámbito **Local** y continúa hacia **Enclosing**, **Global** y finalmente **Built-in**, hasta encontrar la variable solicitada.

6.1.1 Ejemplo de búsqueda de variables según la regla LEGB

En el siguiente ejemplo, la variable **X** no existe dentro de la función, por lo que Python la busca en el ámbito global.

```

[47]: # =====
# Regla LEGB
# Ejemplo de búsqueda de variables
# =====

```

```

# Variable global.
X = 99

# Definimos una función.
def func(Y):

    # La variable X no existe localmente,
    # por lo que Python la busca en el
    # ámbito global.
    Z = X + Y

    return Z

# Ejecutamos la función.
print(func(1))

```

100

6.1.2 Variables locales y variables globales

Cuando una variable se define dentro de una función utilizando el mismo nombre que una variable global, ambas variables son independientes. La variable local oculta temporalmente a la variable global únicamente dentro de la función.

```

[48]: # =====
# Variable local y variable global
# =====

# Variable global.
X = 1

# Definimos una función.
def func():

    # Esta variable es local y no modifica
    # la variable global.
    X = 2

    print("Dentro de la función:", X)

# Ejecutamos la función.
func()

# Mostramos la variable global.
print("Fuera de la función:", X)

```


Dentro de la función: 2

Fuera de la función: 1

6.1.3 La palabra reservada `global`

Si se desea modificar una variable global desde el interior de una función, es necesario utilizar la palabra reservada `global`. De esta manera, Python entiende que se trabajará con la variable definida en el ámbito global y no con una nueva variable local.

```
[49]: # =====
# Uso de la palabra reservada global
# =====

# Variable global.
X = 1

# Definimos una función.
def func():

    # Indicamos que utilizaremos
    # la variable global.
    global X

    # Modificamos su valor.
    X = 2

# Ejecutamos la función.
func()

# Mostramos el nuevo valor de la variable global.
print(X)
```

2

6.1.4 Conclusión

La regla **LEGB** define el orden en que Python busca las variables durante la ejecución de un programa. Comprender este mecanismo permite evitar errores relacionados con el alcance de las variables y facilita la escritura de funciones más claras y mantenibles.

Cuando una variable debe modificarse desde una función y pertenece al ámbito global, se debe utilizar la palabra reservada `global` para indicar explícitamente esta intención.

6.1.5 Referenciar variables globales

No es necesario utilizar la palabra reservada `global` cuando únicamente se desea leer el valor de una variable global. Esta palabra solo es obligatoria cuando se pretende modificar su contenido.

```
[50]: # =====
# Lectura de variables globales
# =====

# Variables globales.
y, z = 1, 2

# Definimos una función.
def todas_globales():

    # Solo necesitamos declarar como global
    # la variable que será modificada.
    global x

    # y y z pueden utilizarse directamente
    # porque únicamente se leen.
    x = y + z

# Ejecutamos la función.
todas_globales()

# Mostramos las variables globales.
print(x)
print(y)
print(z)
```

3

1

2

6.1.6 Scope de una función anidada

Una función definida dentro de otra puede acceder a las variables del ámbito que la contiene. Este ámbito recibe el nombre de **Enclosing Scope** dentro de la regla LEGB.

```
[51]: # =====
# Enclosing Scope
# =====

# Variable global.
X = 77

# Función externa.
def f1():

    # Variable del ámbito enclosing.
    X = 88
```

```

# Función interna.
def f2():

    # Python encuentra primero la variable
    # del ámbito enclosing.
    print(X)

# Ejecutamos la función interna.
f2()

# Ejecutamos la función externa.
f1()

```

88

6.2 Closures

Un **closure** ocurre cuando una función interna recuerda las variables del ámbito que la contiene, incluso después de que la función externa haya finalizado su ejecución.

Gracias a este comportamiento, la función interna puede seguir utilizando dichas variables cuando sea ejecutada posteriormente.

```

[52]: # =====
# Closure
# =====

# Función externa.
def f1():

    # Variable del ámbito enclosing.
    X = 88

    # Función interna.
    def f2():

        # Utiliza la variable X aunque
        # la función externa ya haya terminado.
        print(X)

    # Devolvemos la función sin ejecutarla.
    return f2

# Guardamos la referencia a la función.
accion = f1()

```

```
# Ejecutamos posteriormente la función.
accion()
```

88

6.2.1 La palabra reservada `nonlocal`

La palabra reservada **`nonlocal`** permite modificar una variable que pertenece al ámbito **Enclosing**, es decir, una variable definida en una función externa pero que no es global.

```
[53]: # =====
# Uso de nonlocal
# =====

# Función externa.
def f1():

    # Variable del ámbito enclosing.
    contador = 10

    # Función interna.
    def f2():

        # Indicamos que utilizaremos la variable
        # del ámbito enclosing.
        nonlocal contador

        # Incrementamos su valor.
        contador += 1

        # Mostramos el nuevo valor.
        print(contador)

    # Devolvemos la función.
    return f2

# Creamos el closure.
accion = f1()

# Ejecutamos varias veces la función.
accion()
accion()
accion()
```

11

12

13

6.2.2 Conclusión

Los **closures** permiten que una función conserve el acceso a las variables de su ámbito de creación, incluso después de que la función externa haya terminado. Este comportamiento es muy útil para implementar funciones personalizadas, decoradores y técnicas de programación funcional.

Cuando es necesario modificar una variable perteneciente al ámbito **Enclosing**, Python proporciona la palabra reservada **nonlocal**, evitando la creación de una nueva variable local.

7 1.10 Ejercicios

A continuación se presentan varios ejercicios prácticos para aplicar los conceptos estudiados sobre funciones, argumentos, funciones lambda y manejo de parámetros en Python.

7.1 Ejercicio 1

Enunciado

Escribe una función que reciba como entrada una lista de números y devuelva una nueva lista con el cuadrado de cada uno de sus elementos.

```
[54]: # =====  
# Ejercicio 1  
# Lista con los cuadrados  
# =====  
  
# Definimos la función.  
def cuadrados(lista):  
  
    # Utilizamos comprensión de listas para  
    # calcular el cuadrado de cada elemento.  
    return [x ** 2 for x in lista]  
  
# Probamos la función.  
numeros = [1, 2, 3, 4, 5]  
  
print(cuadrados(numeros))
```

```
[1, 4, 9, 16, 25]
```

7.2 Ejercicio 2

Enunciado

Escribe una función que reciba una cantidad variable de números y devuelva la suma de todos ellos utilizando argumentos arbitrarios.

```
[55]: # =====  
# Ejercicio 2
```

```

# Suma de una cantidad variable de números
# =====

# La función recibe cualquier cantidad de números.
def sumar(*numeros):

    # Sumamos todos los elementos.
    return sum(numeros)

# Probamos la función.
print(sumar(1, 2, 3))

print(sumar(5, 7, 10, 20))

```

6
42

7.3 Ejercicio 3

Enunciado

Escribe una función que reciba una cadena de texto y devuelva la misma cadena invertida.

```

[56]: # =====
# Ejercicio 3
# Invertir una cadena
# =====

# Definimos la función.
def invertir(texto):

    # Utilizamos slicing para invertir
    # la cadena.
    return texto[::-1]

# Probamos la función.
print(invertir("hola"))

```

aloh

7.4 Ejercicio 4

Enunciado

Escribe una función **lambda** que invierta una cadena de texto.

```

[57]: # =====
# Ejercicio 4

```

```
# Lambda para invertir un texto
# =====

# Definimos la función lambda.
invertir = lambda texto: texto[::-1]

# Probamos la función.
print(invertir("hola"))
```

aloh

7.5 Ejercicio 5

Enunciado

Escribe una función que compruebe si un número pertenece a un rango determinado.

```
[58]: # =====
# Ejercicio 5
# Comprobar un rango
# =====

# Definimos la función.
def en_rango(numero, minimo, maximo):

    # Retornamos True o False.
    return minimo <= numero <= maximo

# Probamos la función.
print(en_rango(8, 1, 10))

print(en_rango(20, 1, 10))
```

True

False

7.6 Ejercicio 6

Enunciado

Escribe una función que reciba un número entero positivo y devuelva una lista con sus cinco primeros múltiplos.

```
[59]: # =====
# Ejercicio 6
# Primeros cinco múltiplos
# =====

# Definimos la función.
```

```
def multiplos(numero):

    # Verificamos que el número sea válido.
    if numero <= 0:

        print("Error: el número debe ser mayor que cero.")

        return []

    # Generamos la lista de múltiplos.
    return [numero * i for i in range(1, 6)]

# Probamos la función.
print(multiplos(3))

print(multiplos(-2))
```

[3, 6, 9, 12, 15]

Error: el número debe ser mayor que cero.

[]

7.7 Ejercicio 7

Enunciado

Escribe una función que determine si una lista contiene elementos duplicados.

```
[60]: # =====
# Ejercicio 7
# Verificar duplicados
# =====

# Definimos la función.
def tiene_duplicados(lista):

    # Comparamos el tamaño de la lista
    # con el tamaño del conjunto.
    return len(lista) != len(set(lista))

# Probamos la función.
print(tiene_duplicados([1, 2, 3, 4]))

print(tiene_duplicados([1, 2, 2, 3]))
```

False

True

7.8 Ejercicio 8

Enunciado

Escribe una función **lambda** que determine si una lista contiene elementos duplicados.

```
[61]: # =====  
# Ejercicio 8  
# Lambda para verificar duplicados  
# =====  
  
# Definimos la función lambda.  
duplicados = lambda lista: len(lista) != len(set(lista))  
  
# Probamos la función.  
print(duplicados([1, 2, 3]))  
  
print(duplicados([1, 1, 2]))
```

False

True

7.9 Ejercicio 9

Enunciado

Escribe una función que determine si una cadena de texto es un palíndromo.

```
[62]: # =====  
# Ejercicio 9  
# Verificar un palíndromo  
# =====  
  
# Definimos la función.  
def es_palindromo(texto):  
    # Comparamos el texto con su versión invertida.  
    return texto == texto[::-1]  
  
# Probamos la función.  
print(es_palindromo("reconocer"))  
  
print(es_palindromo("python"))
```

True

False